



Chương 1 – Tổng quan

Lý thuyết automata và ứng dụng (AUTOMATA THEORY AND ITS APPLICATIONS)

Giảng viên: TS. Vũ Đức Lung

Email: lungvd@uit.edu.vn

Khoa KTM

Vũ Đức Lung

1



Automat là gì?

- ☐ Automat theory is the study of abstract computing devices, or “machines”
- ☐ Lý thuyết automata là lý thuyết làm nền tảng cho việc hiểu các mô hình tự động mà điển hình nhất là các máy tính số ngày nay
- ☐ Sự liên quan giữa lý thuyết automata và ngôn ngữ hình thức

2



Tại sao học automata?

- ☐ Thăm dò ý kiến tại một số trường ĐH danh tiếng như trường Stanford với những sinh viên sau tốt nghiệp 5 năm:
Kiến thức môn học nào được sử dụng nhiều nhất trong công việc?
- ☐ Ngoài các môn cơ bản thì môn này được đánh giá cao trong các môn học tự chọn

3



Các ứng dụng chủ yếu

- ☐ Xây dựng ngôn ngữ lập trình và các trình dịch
 - Bộ phân tích từ vựng và cú pháp trong các trình biên dịch
 - Xử lý chuỗi trong ngôn ngữ lập trình và ngôn ngữ tự nhiên
 - Dịch từ ngôn ngữ này sang ngôn ngữ khác: từ ngôn ngữ cấp cao sang cấp thấp, từ tiếng Anh qua tiếng Pháp,...
- ☐ Thiết kế các hệ thống số
 - Mạch tuần tự
 - Mạch đếm
 - Máy trạng thái
- ☐ Phần mềm khai phá dữ liệu web, doc, chống đạo văn
- ☐ Các giao thức truyền thông, truyền tin mật

4



Các kiến thức yêu cầu

- ☐ Đặt nền tảng trên lý thuyết tập hợp
- ☐ Lý thuyết đồ thị
- ☐ Kỹ thuật chứng minh qui nạp, chứng minh phản chứng

5



Yêu cầu và tính điểm

- ☐ Bài tập kiểm tra trên lớp: 20%
 - Bài kiểm tra 1: 10%
 - Bài kiểm tra 2: 10%
- ☐ Thi giữa kỳ: 20%
- ☐ Báo cáo seminar: 30%
 - Thành lập các nhóm 2-3 sinh viên
 - Buổi học 2-4 đăng ký đề tài
 - Báo cáo vào 2 tuần cuối của HK
 - Điểm: Báo cáo + trình bày
- ☐ Thi cuối kỳ: 30%

6



Tài liệu tham khảo

- ❑ Hồ Văn Quân. Lý thuyết automata và ngôn ngữ hình thức.
- ❑ Đọc thêm:
 - Hopcroft, Motwani, Ullman. *Automata Theory, Languages, and Computation* 2nd Edition.
 - Bakhadyr Khousainov. Automata theory and its application. Free reading: <http://books.google.com> (Ai tìm được phiên bản đầy đủ share + bonus)
- ❑ Tài liệu học tập:
 - <https://sites.google.com/site/vdlung/automat>

7



NỘI DUNG

- ❑ Bô túc toán
- ❑ Ngôn ngữ và biểu diễn ngôn ngữ
- ❑ Automata hữu hạn và biểu thức chính quy
- ❑ Văn phạm chính quy và các tính chất
- ❑ Văn phạm phi ngữ cảnh
- ❑ Automata đẩy xuống
- ❑ Máy Turing
- ❑ Ứng dụng thiết kế số

Khoa KTMT

Vũ Đức Lung

8



Bô túc toán

Nội dung:

- Tập hợp
- Quan hệ
- Phép chứng minh quy nạp
- Đồ thị và cây

9



Tập hợp (Set)

Ví dụ:

- $D = \{\text{Mon, Tue, Wed, Thu, Fri, Sat, Sun}\}$

Phần tử

- Tập các đối tượng rời rạc
- Không trùng lặp

Định nghĩa:

- Tập hợp là tập các đối tượng không có sự lặp lại

10



Ký hiệu tập hợp

Liệt kê phần tử:

- $D = \{1, 2, 3\}$

Đặc tả tính chất đặc trưng:

- $D = \{x \mid x \text{ là một ngày trong tuần}\}$

11



Một số dạng tập hợp đặc biệt

Tập rỗng:

- Ký hiệu: $\emptyset$ hoặc $\{\}$

Tập hợp con:

- Ký hiệu: $A \subset B$ (Ngược lại: $A \not\subset B$)
- $\{1, 2, 4\} \subset \{1, 2, 3, 4, 5\}$
- $\{2, 4, 6\} \not\subset \{1, 2, 3, 4, 5\}$

12



Một số dạng tập hợp đặc biệt

Tập hợp bằng nhau:

- Ký hiệu: $A = B$ (Ngược lại: $A \neq B$)
- $\{1, 2\} = \{2, 1\}$ nhưng $\{1, 2, 3\} \neq \{2, 1\}$

Tập lũy thừa:

- Ký hiệu: 2^A
- $A = \{1, 2, 3\}$ thì $2^A = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{2, 3\}, \{3, 1\}, \{1, 2, 3\}\}$

13



Các phép toán trên tập hợp

Phần bù (complement):

- $A' = \{x \mid x \notin A\}$

Phép hợp (Union):

- $A \cup B = \{x \mid x \in A \text{ hoặc } x \in B\}$

Phép giao (intersection):

- $A \cap B = \{x \mid x \in A \text{ và } x \in B\}$

14



Các phép toán trên tập hợp

Phép trừ (difference):

- $A \setminus B = \{x \mid x \in A \text{ nhưng } x \notin B\}$

Tích Đề các:

- $A \times B = \{(a, b) \mid a \in A \text{ và } b \in B\}$

15



Các phép toán trên tập hợp

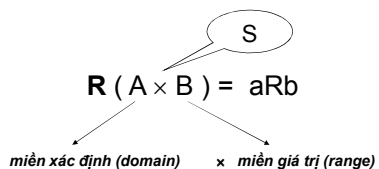
Ví dụ: cho $A = \{1, 2\}$ và $B = \{2, 3\}$

- $A \cup B = \{1, 2, 3\}$
- $A \cap B = \{2\}$
- $A \setminus B = \{1\}$
- $A \times B = \{(1, 2), (1, 3), (2, 2), (2, 3)\}$
- $2^A = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$

16



Quan hệ



17



Quan hệ

Ví dụ: cho $S = \{0, 1, 2, 3\}$

- Quan hệ 'thứ tự nhỏ hơn'
 $L = \{(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)\}$
- Quan hệ 'bằng'
 $E = \{(0, 0), (1, 1), (2, 2), (3, 3)\}$
- Quan hệ 'chẵn lẻ'
 $P = \{(0, 0), (1, 1), (2, 2), (3, 3), (0, 2), (2, 0), (1, 3), (3, 1)\}$

18



Các tính chất của quan hệ

Phản xạ (reflexive): nếu aRa là đúng với $\forall a \in S$

Đối xứng (symmetric): nếu aRb thì bRa

Bắc cầu (transitive): nếu aRb và bRc thì aRc

Ví dụ: ở ví dụ trên

- L không là quan hệ phản xạ trên S vì $(0,0) \notin L$ hay đối xứng trên S vì $(0,1) \in L$ nhưng $(1,0) \notin L$
- E và P mang tính phản xạ, đối xứng và bắc cầu

19



Quan hệ tương đương

Quan hệ tương đương = Quan hệ phản xạ, đối xứng và bắc cầu

Ví dụ:

- E và P là quan hệ tương đương
- L không là quan hệ tương đương

20



Lớp tương đương

Nếu R là quan hệ tương đương trên S thì R phân hoạch S thành các lớp tương đương không rỗng và rời nhau: $S = S_1 \cup S_2 \cup \dots$

Tính chất:

- $S_i \cap S_j = \emptyset$
- Nếu a, b cùng thuộc S_i thì aRb đúng
- Nếu $a \in S_i$ và $b \in S_j$ thì aRb sai

Ví dụ: P có 2 lớp tương đương $\{0, 2\}$ và $\{1, 3\}$

21



Bao đóng của quan hệ

P-closure = quan hệ nhỏ nhất thỏa các tính chất trong P

Bao đóng bắc cầu R^+ :

- Nếu $(a,b) \in R$ thì $(a,b) \in R^+$
- Nếu $(a,b) \in R^+$ và $(b,c) \in R$ thì $(a,c) \in R^+$
- Không còn gì thêm trong R^+

Bao đóng phản xạ và bắc cầu R^* :

- $R^* = R^+ \cup \{(a, a) \mid a \in S\}$

22



Bao đóng của quan hệ

Ví dụ: $R = \{(1, 2), (2, 2), (2, 3)\}$ trên $S = \{1, 2, 3\}$

- $R^+ = \{(1, 2), (2, 2), (2, 3), (1, 3)\}$
- $R^* = \{(1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)\}$

23



Nguyên lý quy nạp

Bước 1 (cơ sở quy nạp): chứng minh $P(0)$

Bước 2 (giả thiết quy nạp): giả sử $P(n-1)$

Bước 3 (quy nạp): $P(n-1) \Rightarrow P(n), \forall n \geq 1$.

Ví dụ: chứng minh

$$\sum_{i=0}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

24



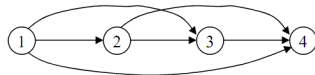
Đồ thị có hướng (Directed graph)

Đồ thị $G = (V, E)$

- V : tập các đỉnh (nút)
- E : tập các cung có hướng $v \rightarrow w$

Ví dụ: đồ thị $G = (V, E)$

- $V = \{ 1, 2, 3, 4 \}$
- $E = \{ i \rightarrow j \mid i < j \}$



25



Cây (Trees)

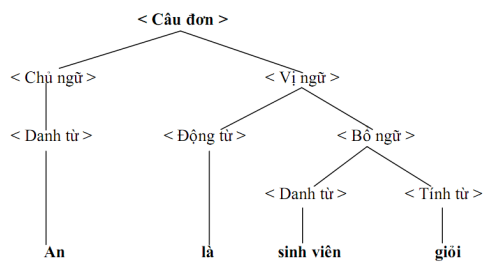
Cây: là đồ thị có hướng

- 1 nút gốc
- Nút trung gian (nút trong)
- Nút lá: không dẫn ra nút con
- Thứ tự duyệt trên cây: **trái $\rightarrow$ phải**

26



Cây minh họa một câu đơn



Khoa KTMT

Vũ Đức Lung

27



CÂU HỎI VÀ BÀI TẬP CHƯƠNG I

Khoa KTMT

Vũ Đức Lung

28



Chương 02 – Ngôn ngữ và biểu diễn ngôn ngữ

Giảng viên: TS. Vũ Đức Lung
Email: lungvd@uit.edu.vn

Khoa KTM

Vũ Đức Lung

1



Nội dung:

- Khái niệm ngôn ngữ
- Cách biểu diễn ngôn ngữ
- Văn phạm
- Sự phân lớp văn phạm

2



Ký hiệu, bộ chữ cái, chuỗi

Ký hiệu (symbol): là một thực thể trừu tượng mà ta không định nghĩa được một cách hình thức

- Các chữ cái a, b, c ... hoặc các số 1, 2, 3 ...

Bộ chữ cái (alphabet): Σ

- Là một tập (không rỗng) các ký hiệu nào đó
- Bộ chữ cái Latin $\{A, B, C, \dots, a, b, c, \dots, z\}$

Chuỗi (string): một chuỗi (hay một từ - word) trên bộ chữ cái Σ

- Là một dãy hữu hạn các ký hiệu của Σ
- Một ký hiệu có thể xuất hiện nhiều lần

3



Chuỗi

Độ dài chuỗi: là số các ký hiệu tạo thành chuỗi

- $|abca| = 4$

Chuỗi rỗng: ký hiệu ϵ , là chuỗi không có ký hiệu nào

- $|\epsilon| = 0$

Chuỗi con: chuỗi v là chuỗi con của w nếu v được tạo bởi các ký hiệu liên tiếp nhau trong chuỗi w.

- Chuỗi 10 là chuỗi con của chuỗi 01001

Chuỗi tiền tố: là chuỗi con bất kỳ nằm ở đầu chuỗi

Chuỗi hậu tố: là chuỗi con bất kỳ nằm ở cuối chuỗi

- Chuỗi abc có các tiền tố a, ab, abc
- Chuỗi 0246 có các hậu tố 6, 46, 246, 0246

4



Chuỗi

Chuỗi nối kết (ghép): là chuỗi được tạo thành bằng cách viết chuỗi thứ nhất, sau đó viết chuỗi thứ hai, ...

- Nối ghép của chuỗi Long và Int là LongInt
- Nối kết của chuỗi rỗng: $\epsilon w = w\epsilon = w$ (với mọi w)
→ ϵ là đơn vị của phép nối kết

Chuỗi đảo ngược: của chuỗi w, ký hiệu w^R , là chuỗi w được viết theo thứ tự ngược lại.

$$w = abcd \rightarrow w^R = dcba \quad \epsilon^R = \epsilon$$

5



Ngôn ngữ (Languages)

Tổng quan về ngôn ngữ:

- Ngôn ngữ tự nhiên: tiếng Việt, tiếng Anh, ...
- Ngôn ngữ lập trình: Pascal, C/C++, ...
- Là tập hợp các câu theo cấu trúc quy định nào đó
- Biểu thị các ý nghĩ, các sự kiện hay các khái niệm
- Bao gồm một tập các ký hiệu và các quy tắc để vận dụng chúng

6



Ngôn ngữ (Languages)

Một ngôn ngữ (hình thức) L là một **tập hợp các chuỗi** của các ký hiệu từ một bộ chữ cái Σ nào đó.

Σ^* và Σ^+ :

- Σ^* : tập hợp tất cả các chuỗi con, kể cả chuỗi rỗng ϵ , sinh ra từ bộ chữ cái Σ .
- Σ^+ : tập hợp tất cả các chuỗi con, ngoại trừ chuỗi rỗng ϵ , sinh ra từ bộ chữ cái Σ .
- $\Sigma^* = \Sigma^+ + \{\epsilon\}$ $\Sigma^+ = \Sigma^* - \{\epsilon\}$
- $\Sigma = \{0,1\}$ thì:
 - $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$
 - $\Sigma^+ = \{0, 1, 00, 01, 10, 11, 000, \dots\}$
 - Chuỗi 010210 $\notin \Sigma^*$ vì có số 2 $\notin \Sigma$

7



Các phép toán trên ngôn ngữ

Phép bao đóng (closure): thành lập một ngôn ngữ bằng cách kết nối các chuỗi (với số lượng bất kỳ) các chuỗi của một ngôn ngữ L cho trước

❖ Bao đóng Kleene: $L^* = \bigcup_{i=0}^{\infty} L^i$

❖ Bao đóng dương (positive): $L^+ = \bigcup_{i=1}^{\infty} L^i$

Chú ý: $L^* = L^*L = LL^*$ $L^* = L^+ \cup \{\epsilon\}$

Ví dụ: cho $L = \{a, ba\}$

- $L^2 = \{aa, aba, baa, baba\}$
- $L^3 = \{aaa, aaba, abaa, ababa, baaa, baaba, babaa, bababa\}$
- $L^* = \{\epsilon, a, ba, aa, aba, baa, baba, aaa, aaba, \dots\}$

8



Biểu diễn ngôn ngữ

Liệt kê chuỗi: $L = \{aa, aba, baa, baba\}$

Mô tả đặc điểm chủ yếu: $L = \{a^i \mid i \text{ là số nguyên tố}\}$

Biểu diễn thông qua văn phạm và automata:

- Cho phép biểu diễn ngôn ngữ một cách tổng quát
- Văn phạm: cơ chế sản sinh ra mọi chuỗi của ngôn ngữ
- Automata: cơ chế cho phép đoán nhận một chuỗi bất kỳ có thuộc một ngôn ngữ L hay không

9



Định nghĩa văn phạm

Theo từ điển, văn phạm là một tập các quy tắc về cấu tạo từ và các quy tắc về cách thức liên kết từ lại thành câu

Định nghĩa: văn phạm cấu trúc G là một hệ thống gồm 4 thành phần $G(V, T, P, S)$

- V (variables): tập các biến (VD: $A, B, C, \dots$)
- T (terminal): tập các ký hiệu kết thúc ($V \cap T = \emptyset$) (VD: $a, b, c, \dots, w, x, y, \dots$)
- P (production): tập luật sinh, dạng $\alpha \rightarrow \beta$ với $\alpha, \beta \in (V \cup T)^*$
- S (start): ký hiệu bắt đầu ($S \in V$)

10



Định nghĩa văn phạm

Dẫn xuất trực tiếp: nếu $\alpha \rightarrow \beta$ là một luật sinh thì

$$\gamma \alpha \delta \Rightarrow \gamma \beta \delta$$

Dẫn xuất gián tiếp: nếu các chuỗi $\alpha_1, \alpha_2, \dots, \alpha_m \in \Sigma^*$ và $\alpha_1 \Rightarrow \alpha_2, \alpha_2 \Rightarrow \alpha_3, \dots, \alpha_{m-1} \Rightarrow \alpha_m$ thì α_m có thể được dẫn xuất từ α_1

$$\alpha_1 \Rightarrow^* \alpha_m$$

Ngôn ngữ L sinh bởi văn phạm G :

$$L(G) = \{w \mid w \in T^* \text{ và } S \Rightarrow^* w\}$$

Văn phạm tương đương: là 2 văn phạm sinh ra cùng một ngôn ngữ (G_1 tương đương $G_2 \Leftrightarrow L(G_1) = L(G_2)$)

11



Phân cấp Chomsky trên văn phạm

Bằng cách áp đặt một số quy tắc hạn chế trên các luật sinh, Noam Chomsky đề nghị một hệ thống phân loại các văn phạm dựa vào tính chất của các luật sinh.

Loại 0 – Văn phạm không hạn chế (Unrestricted Grammar): không cần thỏa điều kiện ràng buộc nào trên tập các luật sinh

Loại 1 – Văn phạm cảm ngữ cảnh (CSG – Context Sensitive Grammar): nếu văn phạm G có các luật sinh dạng $\alpha \rightarrow \beta$ và $|\beta| \geq |\alpha|$

Loại 2 – Văn phạm phi ngữ cảnh (CFG – Context-Free Grammar): có luật sinh dạng $A \rightarrow \alpha$ với A là một biến đơn và α là chuỗi các ký hiệu thuộc $(V \cup T)^*$

12



Phân cấp Chomsky trên văn phạm

Loại 3 – Văn phạm chính quy (RG – Regular)

Grammar: có mọi luật sinh dạng tuyến tính phải hoặc tuyến tính trái.

- Tuyến tính phải: $A \rightarrow wB$ hoặc $A \rightarrow w$
- Tuyến tính trái: $A \rightarrow Bw$ hoặc $A \rightarrow w$

Với A, B là các biến đơn, w là chuỗi ký hiệu kết thúc (có thể là rỗng)

Nếu ký hiệu L_0, L_1, L_2, L_3 là các ngôn ngữ được sinh ra bởi văn phạm loại 0, 1, 2, 3, ta có:

$$L_3 \subset L_2 \subset L_1 \subset L_0$$

13



CÂU HỎI VÀ BÀI TẬP CHƯƠNG 02

Khoa KTMT

Vũ Đức Lung

14

Chương 3: Automata hữu hạn & Biểu thức chính quy

Nội dung:

- Khái niệm DFA & NFA
- Sự tương đương giữa DFA & NFA
- Biểu thức chính quy
- Các tính chất của tập chính quy

1

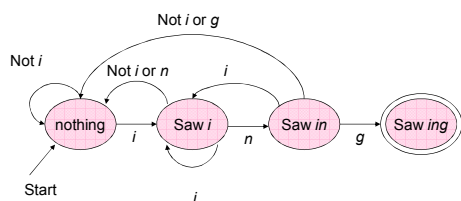
Khái niệm Automat hữu hạn (Finite Automata)

- FA là tập các trạng thái hữu hạn với các qui tắc để chuyển đổi từ một trạng thái này sang trạng thái khác.
- Ứng dụng nguyên thủy của FA là mạch chuyển trạng thái tuần tự (mạch tuần tự), trong đó trạng thái là cài đặt một tập các bits trong các flip-flop.
- Ngày nay được ứng dụng rộng rãi.
- Biểu diễn đơn giản nhất là “Đồ thị chuyển trạng thái” hay “sơ đồ dịch”

2

VD 1: Nhận biết chuỗi kết thúc “ing”

Biểu diễn FA bằng sơ đồ dịch



3

Chuyển Automata thành Code

- In C/C++, tạo đoạn code cho từng trạng thái. Code sẽ có dạng:

1. Đọc ký tự nhập.
2. Tính toán để ra quyết định trạng thái kế.
3. Nhảy đến đoạn code của trạng thái kế đó.

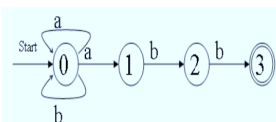
```
2: /* State "Saw i" */
   c = getNextInput();
   if (c == 'n') goto 3;
   else if (c == 'i') goto 2;
   else goto 1;
3: /* State "Saw in" */
   . . .
```

4

Biểu diễn FA bằng bảng truyền

Thí dụ: Cho NFA:

Tập trạng thái $S = \{0, 1, 2, 3\}$; $\Sigma = \{a, b\}$; Trạng thái bắt đầu $S_0 = 0$;
Tập trạng thái kết thúc $F = \{3\}$.



NFA nhận dạng $(a|b)^*abb$

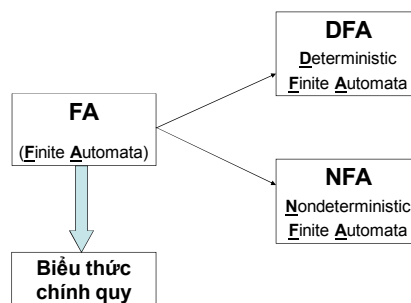
Bảng truyền

Trạng thái	Ký hiệu nhập	
	a	b
0	{0, 1}	{0}
1	-	{2}
2	-	{3}

Khoa KTMT - UIT

TS

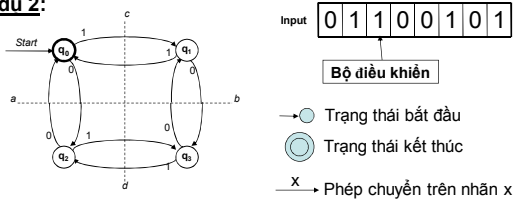
Phân loại FA



6

Automata hữu hạn đơn định (DFA)

Ví dụ 2:



$M = (Q, \Sigma, \delta, q_0, F)$

- Q : tập hữu hạn các trạng thái ($p, q, \dots$)
- Σ : bộ chữ cái nhập ($a, b, \dots; w, x, y, \dots$)
- δ : hàm chuyển, ánh xạ: $Q \times \Sigma \rightarrow Q$
- $q_0 \in Q$: trạng thái bắt đầu.
- $F \subseteq Q$: tập các trạng thái kết thúc.

Mở rộng hàm chuyển trạng thái

- $\delta(q, \epsilon) = q$
- $\delta(q, wa) = \delta(\delta(q, w), a)$ với $\forall w, a$

Ngôn ngữ được chấp nhận:

$$L(M) = \{ x \mid \delta(q_0, x) \in F \}$$

Ví dụ: chuỗi nhập $w = 110101$

- $\delta(q_0, 1) = q_1$
- $\delta(q_0, 11) = \delta(q_1, 1) = q_0$
- $\delta(q_0, 110) = \delta(q_1, 0) = \delta(q_0, 0) = q_2$
- $\delta(q_0, 1101) = \delta(q_2, 1) = \delta(q_0, 01) = \delta(q_2, 1) = q_3$
- $\delta(q_0, 11010) = \dots = \delta(q_3, 0) = q_1$
- $\delta(q_0, 110101) = \dots = \delta(q_1, 1) = q_0 \in F$

Ngôn ngữ chính quy

8

Giải thuật hình thức

- Mục đích:** kiểm tra một chuỗi nhập x có thuộc ngôn ngữ $L(M)$ được chấp nhận bởi automata M .
- Input:** chuỗi nhập x
- Output:** câu trả lời 'YES' hoặc 'NO'
- Giải thuật:**

```

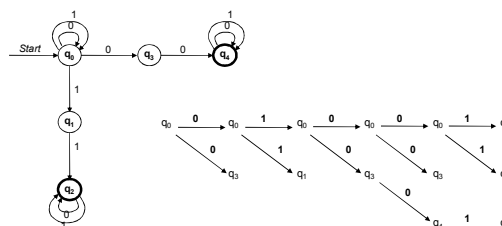
q := q0 ;
c := nextchar ; {c là ký hiệu nhập được đọc tiếp theo}
While c <> $ do
  begin
    q := δ(q, c) ;
    c := nextchar ;
  end
If (q in F) then write("YES") else write("NO") ;

```

9

Automata hữu hạn không đơn định (NFA)

- Ví dụ:** cho automata M (hình vẽ) và xét chuỗi nhập **01001**



Nhận xét:

- Ứng với một trạng thái và một ký tự nhập, có thể có không, một hoặc nhiều phép chuyển trạng thái.
- DFA là một trường hợp đặc biệt của NFA

10

Định nghĩa NFA

$M = (Q, \Sigma, \delta, q_0, F)$

- Q : tập hữu hạn các trạng thái.
- Σ : bộ chữ cái nhập.
- δ : hàm chuyển ánh xạ $Q \times \Sigma \rightarrow 2^Q$
- $q_0 \in Q$: trạng thái bắt đầu.
- $F \subseteq Q$: tập các trạng thái kết thúc.

Chú ý: khái niệm $\delta(q, a)$ là tập hợp tất cả các trạng thái p sao cho có phép chuyển từ trạng thái q trên nhãn a .

Hàm chuyển trạng thái mở rộng:

- $\delta(q, \epsilon) = \{q\}$
- $\delta(q, wa) = \{ p \mid \text{có một trạng thái } r \text{ trong } \delta(q, w) \text{ mà } p \in \delta(r, a) \}$
 $= \delta(\delta(q, w), a)$
- $\delta(P, w) = \bigcup_{q \in P} \delta(q, w)$ với $\forall P \subseteq Q$

11

Ví dụ về NFA

Ví dụ: xét chuỗi nhập $w = 01001$ và NFA đã cho ở trên

- $M(\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \delta, q_0, \{q_4\})$

δ	Input	
Trạng thái	0	1
q_0	$\{q_0, q_3\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\{q_2\}$	$\{q_2\}$
q_3	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$

- $\delta(q_0, 0) = \{q_0, q_3\}$
- $\delta(q_0, 01) = \delta(\delta(q_0, 0), 1)$
 $= \delta(\{q_0, q_3\}, 1) = \delta(q_0, 1) \cup \delta(q_3, 1) = \{q_0, q_1\}$
- $\delta(q_0, 010) = \{q_0, q_3\}$
- $\delta(q_0, 0100) = \{q_0, q_3, q_4\}$
- $\delta(q_0, 01001) = \{q_0, q_1, q_4\}$

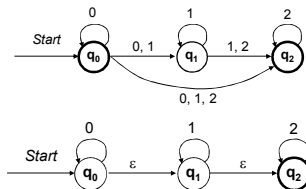
Do $q_4 \in F$ nên $w = 01001 \in L(M)$

12



NFA với ϵ - dịch chuyển (NFA ϵ)

Ví dụ: xây dựng NFA chấp nhận chuỗi 0*1*2*



Định nghĩa: NFA ϵ $M(Q, \Sigma, \delta, q_0, F)$

- δ : hàm chuyển ảnh xạ $Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$
- Khái niệm $\delta(q, a)$ là tập hợp các trạng thái p sao cho có phép chuyển nhân a từ q tới p , với $a \in (\Sigma \cup \{\epsilon\})$

13



Sự tương đương giữa DFA & NFA

Định lý 1: Nếu L là tập được chấp nhận bởi một NFA thì tồn tại một DFA chấp nhận L .

Giả sử NFA $M = (Q, \Sigma, \delta, q_0, F)$ chấp nhận L

Ta xây dựng DFA $M' = (Q', \Sigma, \delta', q_0', F')$ chấp nhận L

- $Q' = 2^Q$. Một phần tử trong Q' được ký hiệu là $[q_0, q_1, \dots, q_i]$ với $q_0, q_1, \dots, q_i \in Q$
- $q_0' = [q_0]$
- F' là tập hợp các trạng thái của Q' có chứa ít nhất một trạng thái kết thúc trong tập F của M
- Hàm chuyển $\delta'([q_1, q_2, \dots, q_i], a) = [p_1, p_2, \dots, p_j]$ nếu và chỉ nếu $\delta(q_1, a) = \{p_1\}, \delta(q_2, a) = \{p_2\}, \dots, \delta(q_i, a) = \{p_j\}$

14



Ví dụ về sự tương đương giữa DFA & NFA

Ví dụ: NFA $M(\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$ với hàm chuyển $\delta(q_0, 0) = \{q_0, q_1\}, \delta(q_0, 1) = \{q_1\}, \delta(q_1, 0) = \emptyset, \delta(q_1, 1) = \{q_0, q_1\}$

Ta sẽ xây dựng DFA tương đương $M' (Q', \{0, 1\}, \delta', [q_0], F')$

- $Q' = \{\emptyset, [q_0], [q_1], [q_0, q_1]\}$
- $F' = \{[q_1], [q_0, q_1]\}$
- Hàm chuyển δ'
 - $\delta'(\emptyset, 0) = \delta'(\emptyset, 1) = \emptyset$
 - $\delta'([q_0], 0) = [q_0, q_1]$
 - $\delta'([q_0], 1) = [q_1]$
 - $\delta'([q_1], 0) = \emptyset$
 - $\delta'([q_1], 1) = [q_0, q_1]$
 - $\delta'([q_0, q_1], 0) = [q_0, q_1]$
 - $\delta'([q_0, q_1], 1) = [q_0, q_1]$

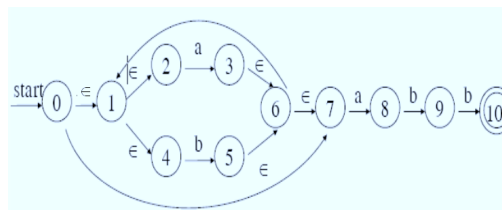
Xem VD 2.12, 2.13 trong [1]

15



Chuyển NFA thành DFA

- Xây dựng DFA tương đương cho NFA sau (NFA nhận dạng $(a|b)^*abb$).



16



Tìm các trạng thái cho DFA

Các bước thực hiện:

- Tính ϵ -closure(0) = $\{0, 1, 2, 4, 7\} = A$ // Những trạng thái trên NFA có từ "0" đi mà có sự truyền rộng ϵ
 - Tính ϵ -closure(Move(A, a)) = ϵ -closure(3, 8) = $\{1, 2, 3, 4, 6, 7, 8\} = B$
Trong đó: Move(A, a) = (3, 8)
 - Tính ϵ -closure(Move(A, b)) = ϵ -closure(5) = $\{1, 2, 4, 5, 6, 7\} = C$
- Lặp lại các bước này cho tất cả các tập B, C, ... cho đến khi không còn tập nào

Kết quả:

A = $\{0, 1, 2, 4, 7\}$
B = $\{1, 2, 3, 4, 6, 7, 8\}$
C = $\{1, 2, 4, 5, 6, 7\}$
D = $\{1, 2, 4, 5, 6, 7, 9\}$
E = $\{1, 2, 4, 5, 6, 7, 10\}$

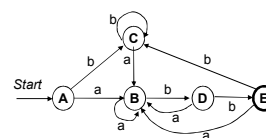
17



Xây dựng DFA từ NFA(ϵ)

- Bảng hàm chuyển

Trạng thái	Ký hiệu nhập	
	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C



- Ký hiệu bắt đầu: $q_0' = A (\leftrightarrow \epsilon$ -CLOSURE(q_0))
- Tập trạng thái kết thúc: $F' = \{E\}$ (vì trong E có chứa trạng thái 10 $\in F$)

18



Sự tương đương giữa NFA_{ϵ} và NFA

Định lý 2: nếu L được chấp nhận bởi một NFA có ϵ -dịch chuyển thì L cũng được chấp nhận bởi một NFA không có ϵ -dịch chuyển.

Giả sử: $NFA_{\epsilon} M(Q, \Sigma, \delta, q_0, F)$ chấp nhận L

Ta xây dựng: $NFA M' = (Q, \Sigma, \delta', q_0, F')$

Với:

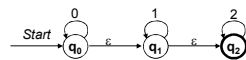
- $F' = F \cup q_0$ nếu ϵ -CLOSURE(q_0) chứa một trạng thái thuộc F.
- Ngược lại, $F' = F$
- $\delta'(q, a) = \delta^*(q, a)$

19



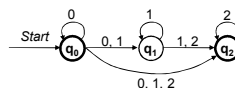
Sự tương đương giữa NFA_{ϵ} và NFA

Ví dụ:



Xây dựng NFA tương đương $M' = (Q, \Sigma, \delta', q_0, F')$

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1, 2\}$
- Trạng thái bắt đầu: q_0
- $F' = \{q_0, q_2\}$
- Hàm chuyển δ'



δ'	Inputs		
Trạng thái	0	1	2
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	$\emptyset$	$\{q_1, q_2\}$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$	$\{q_2\}$

20



Biểu thức chính quy (RE)

Vài ví dụ:

- 00^* : là biểu thức chính quy biểu diễn tập $\{00\}$
- $(0+1)^*$: tập hợp tất cả các chuỗi số 0 và số 1, kể cả chuỗi rỗng $= \{\epsilon, 0, 1, 00, 01, 10, 11, 010, 011, 0010, \dots\}$
- $(0+1)^*011$: ký hiệu cho tất cả các chuỗi 0, 1 tận cùng bởi 011 $= \{011, 0011, 1011, 00011, 11011, \dots\}$
- $(0+1)^*00(0+1)^*$: tập hợp tất cả các chuỗi 0, 1 có ít nhất hai số 0 liên tiếp $= \{00, 000, 100, 0000, 0001, 1000, 1001, 011001, \dots\}$
- $(0+1)^*(1+10)^*$: tất cả các chuỗi không có hai số 0 liên tiếp $= \{\epsilon, 0, 01, 010, 1, 10, 01010, 0111, \dots\}$
- $0^*1^*2^*$: $\{\epsilon, 0, 1, 2, 01, 02, 12, 012, 0012, 0112, \dots\}$
- $00^*11^*22^*$: tất cả các chuỗi trong tập $0^*1^*2^*$ với ít nhất một ký hiệu 0, 1 và 2 $\leftrightarrow$ viết gọn thành $0^*1^*2^*$

21



Biểu thức chính quy (RE)

Định nghĩa: cho Σ là một bộ chữ cái. BTCQ trên Σ là các tập hợp mà chúng mô tả được định nghĩa đệ quy như sau:

- $\emptyset$ là BTCQ ký hiệu cho tập rỗng
- ϵ là BTCQ ký hiệu cho tập $\{\epsilon\}$
- $\forall a \in \Sigma, a$ là BTCQ ký hiệu cho tập $\{a\}$
- Nếu r và s là các BTCQ ký hiệu cho các tập hợp R và S thì $(r + s)$, (rs) và (r^*) là các BTCQ ký hiệu cho các tập hợp $R \cup S$, RS và R^* tương ứng

Thứ tự ưu tiên:

Phép bao đóng > Phép nối kết > Phép hợp

Ví dụ:

- Biểu thức $((0(1^*)) + 1)$ có thể viết là 01^*+1

22



Tính chất đại số của BTCQ

Phép hợp:

- $r + \emptyset = \emptyset + r = r$
- $r + r = r$
- $r + s = s + r$
- $(r + s) + t = r + (s + t) = r + s + t$

Phép nối kết:

- $r\epsilon = \epsilon r = r$
- $r\emptyset = \emptyset r = \emptyset$
- $(r + s)t = rt + st$
- $r(s + t) = rs + rt$

Phép bao đóng:

- $\epsilon^* = \epsilon$
- $\emptyset^* = \emptyset$
- $r^*r^* = r^*$
- $(r^*)^* = r^*$
- $r^* = \epsilon + r + r^2 + \dots + r^k + \dots$
- $r^* = \epsilon + r^+$
- $(\epsilon + r)^* = (\epsilon + r^+)^* = r^*$
- $r^*r = r$ $r^*r^* = r^*$

Tổng hợp:

- $(r^* + s^*)^* = (r^*s^*)^* = (r + s)^*$
- $(rs)^*r = r(sr)^*$
- $(r^*s)^*r^* = (r + s)^*$

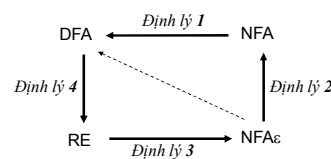
23



Sự tương đương giữa NFA_{ϵ} và BTCQ

Định lý 3: nếu r là BTCQ thì tồn tại một NFA với ϵ -dịch chuyển chấp nhận $L(r)$

Định lý 4: Nếu L được chấp nhận bởi một DFA, thì L được ký hiệu bởi một BTCQ



24



Từ biểu thức chính quy đến NFA

Xây dựng NFA từ biểu thức chính quy

Giải thuật: Xây dựng NFA từ biểu thức chính quy (Cấu trúc Thompson')

Nhập: Biểu thức chính quy r trên Σ

Xuất: NFA nhận dạng ngôn ngữ $L(r)$.

Khoa KTM - UIT

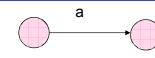
TS. Vũ Đức Lung

25

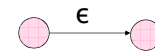


Từ RE đến ϵ -NFA

□ Symbol a :



□ ϵ :



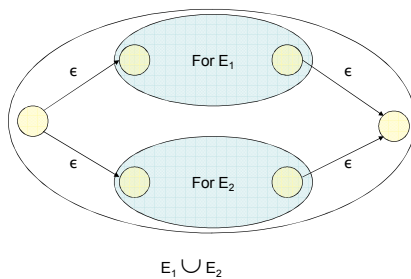
□ $\emptyset$:



26



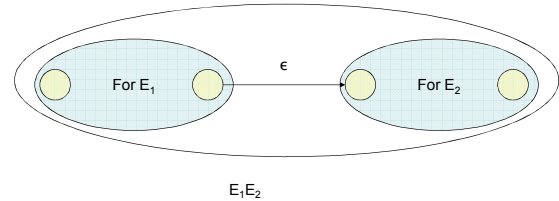
Từ RE đến ϵ -NFA



27



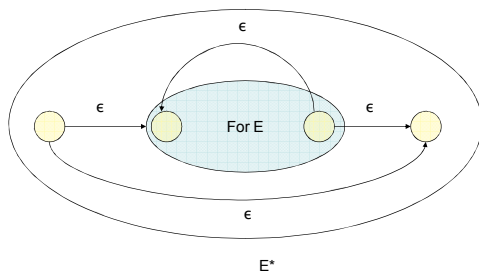
Từ RE đến ϵ -NFA



28



Từ RE đến ϵ -NFA



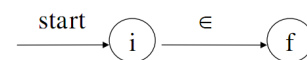
29



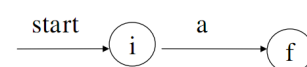
Phương pháp

Quy tắc:

1. Với ϵ , xây dựng NFA



2. Với a thuộc Σ , xây dựng NFA



Khoa KTM - UIT

TS. Vũ Đức Lung

30

Phương pháp

Giả sử $N(s)$ và $N(t)$ là NFA cho biểu thức chính quy s và t .

□ Với $s|t$ xây dựng NFA hỗn hợp $N(s|t)$

Khoa KTM - UIT TS. Vũ Đức Lung 31

Phương pháp

Với biểu thức st , xây dựng NFA hỗn hợp $N(st)$

Khoa KTM - UIT TS. Vũ Đức Lung 32

Phương pháp

- Với biểu thức s^* , xây dựng NFA $N(s^*)$

- Biểu thức s thì $N(s)$ là NFA nhận dạng $L(s)$

Khoa KTM - UIT TS. Vũ Đức Lung 33

Automat hữu hạn

□ Automat hữu hạn không tắt định (NFA)

- Cách vẽ NFA cơ bản

b^*	b^+
$(a b)$	$(a b)^+ = (a b)^*(a b)$ $(a E)^* = a^*$

Khoa KTM - UIT TS. Vũ Đức Lung 34

Automat hữu hạn

□ Automat hữu hạn không tắt định (NFA)

Cách vẽ NFA cơ bản

NFA của $ab^*(a|b)^+a$

35

Ví dụ

➤ Dùng giải thuật để xây dựng NFA cho biểu thức chính quy $r = (a|b)^*abb$

36



Mô phỏng NFA

- **Nhập:** NFA gọi là N , chuỗi nhập x . x được kết thúc bằng eof, N có trạng thái bắt đầu s_0 và tập trạng thái kết thúc F .
- **Xuất:** Giải thuật trả lời đúng nếu N chấp nhận x , ngược lại trả lời sai.

37



Giải thuật

```

S =  $\epsilon$ -closure( $\{s_0\}$ )
a = nextchar
While a  $\neq$  eof then
Begin
    S =  $\epsilon$ -closure(move(s,a))
    a = nextchar
End
if  $S \cap F \neq \emptyset$  then write(đúng)
Else write(sai)

```

38



Xây dựng DFA trực tiếp từ biểu thức chính quy

Tìm hiểu 2 giải thuật để tối ưu việc so trùng mẫu được xây dựng từ biểu thức chính quy:

- Xây dựng DFA trực tiếp từ biểu thức chính quy.
- Tối thiểu trạng thái của DFA.

39



Trạng thái quan trọng của NFA

- Trạng thái quan trọng là từ nó có sự truyền khác rỗng. Như vậy nếu hai tập trạng thái có cùng số trạng thái quan trọng thì chúng được đồng nhất.
- NFA được xây dựng theo cấu trúc Thompson có trạng thái kết thúc không có sự truyền ra, như vậy nó không phải là trạng thái quan trọng (nhưng thực sự nó lại rất quan trọng). Để tránh tình trạng này người ta thêm ký hiệu # vào sau biểu thức chính quy, và trạng thái kết thúc có sự truyền trên ký hiệu #.
- Khi xây dựng tập con hoàn tất thì trạng thái nào có sự truyền trên # là trạng thái chấp nhận.

40



Biểu thức chính quy gia tổ

- Biểu thức $r\#$ được gọi là biểu thức chính quy gia tổ. Ký hiệu # không thuộc tập các ký hiệu cơ bản của biểu thức chính quy r .
- Biểu diễn biểu thức chính quy gia tổ bằng cây phân tích, sao cho tên các lá là các ký hiệu cơ bản. Các nút trung gian là các toán tử dạng: cat, or hoặc star.

Vấn phạm của biểu thức chính quy:

$exp \rightarrow exp \mid term$	$exp \rightarrow term$	$term \rightarrow term \cdot factor$
$term \rightarrow factor$	$factor \rightarrow factor^*$	$factor \rightarrow (exp)$
$factor \rightarrow a$	$factor \rightarrow b$	

41



Xây dựng DFA trực tiếp từ biểu thức chính quy

- **Xây dựng DFA trực tiếp từ biểu thức chính quy:**
 - Vẽ cây phân tích
 - Tính các giá trị Nullable, First Post (FP), Last Post (LP).
 - Lập bảng Follow Post (FLP)
 - Tìm tập các trạng thái, bảng truyền
 - Vẽ DFA
- Tối thiểu trạng thái của DFA.

42

Cây phân tích

- Là cây có nút lá là các ký hiệu cơ bản của $r\#$.
- Các nút là các toán tử.
- Các toán tử trên cây phân tích như:
 - ❖ Toán tử kết nối
 - ❖ Toán tử tuyến.
 - ❖ Toán tử bao đóng truyền.

43

Cây phân tích

□ Cách vẽ cây phân tích $r = (a|ba)(a|b)^*ba\#$

44

Cây phân tích

□ Cây phân tích $r = (a|ba)(a|b)^*ba\#$

45

Các quy tắc để tính ba hàm nullable, firstpos, lastpos

Nút n	nullable (n)	firstpos (n)	lastpos (n)
n là nút có tên là ϵ	true		
n là nút có tên là vị trí i	false	{i}	{i}
	$nullable(c_1)$ or $nullable(c_2)$	$firstpos(c_1) \cup firstpos(c_2)$	$lastpos(c_1) \cup lastpos(c_2)$
	$nullable(c_1)$ and $nullable(c_2)$	if $nullable(c_1)$ then $firstpos(c_1) \cup firstpos(c_2)$ else $firstpos(c_1)$	if $nullable(c_2)$ then $lastpos(c_1) \cup lastpos(c_2)$ else $lastpos(c_2)$
	true	$firstpos(c_1)$	$lastpos(c_1)$

46

Các quy tắc để tính ba hàm nullable, firstpos, lastpos

□ **NULLABLE:**

Quy tắc:

- Nút lá là False (F)
- Nút (*) là True (T)
- Nút (|) là True (T) nếu 1 trong 2 nút con là True (T)
- Nút (.) là True (T) nếu cả 2 nút con đều True (T)

□ **FIRST POST (FP), LAST POST (LP):**

Quy tắc:

- Bắt đầu đánh số cho các nút lá theo thứ tự từ 1 lớn dần (tự đánh)
- FP và LP của nút lá bằng chính số của nó
- Nút (|): FP của nó bằng tổng hợp FP của 2 nút con. LP cũng thế
- Nút (*): FP = FP nút con. LP cũng thế
- Nút (.):
 - o FP: Nếu (Nullable nút con bên trái) = F thì (FP của nó) = (FP nút con bên trái). Nếu (Nullable nút con bên trái) = T thì (FP của nó) = tổng hợp (FP cả 2 nút con)
 - o LP: Nếu (Nullable nút con bên phải) = F thì (LP của nó) = (LP nút con bên phải). Nếu (Nullable nút con bên phải) = T thì (LP của nó) = tổng hợp (LP cả 2 nút con)

47

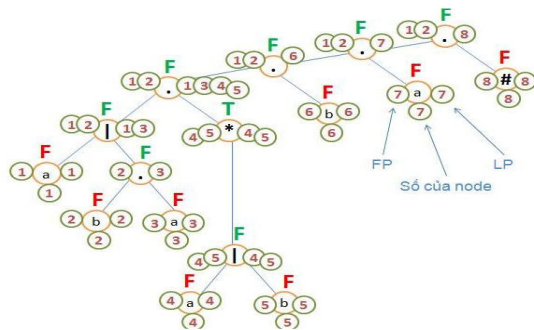
NULLABLE

□ $r = (a|ba)(a|b)^*ba\#$

Giá trị hàm Nullable tại các nút

48

FIRST POST (FP), LAST POST (LP)



49

Các quy tắc tính hàm followpos(n)

➤ Quy tắc:

- Lập bảng FLP bằng cách liệt kê các nốt lá theo hàng dọc (Số thứ tự đã đánh trước)
- Chỉ xét các nốt (.) và (*), không xét nốt (!)
- Cách xét:
 - o Đối với (.): Đưa (tập FP nốt con bên phải) vào (từng giá trị LP của nốt con trái) trong bảng FLP
 - o Đối với (*): Đưa (tập FP của chính nó) vào (từng giá trị LP của chính nó) trong bảng FLP
- Cứ xét hết các nốt cần xét và điền giá trị vào các dòng tương ứng trong bảng FLP

50

Bảng FLP

Bảng FLP

1	4	5	6
2	3		
3	4	5	6
4	4	5	6
5	4	5	6
6	7		
7	8		
8	-		

51

Tìm tập các trạng thái

- Đặt $[A]$ là trạng thái bắt đầu. $[A]$ sẽ chứa FP của ROOT
 $\Rightarrow [A] = \{1, 2\}$

Xét $[A] = \{1,2\}$: Ở đây ta nhìn hình chỉ số các nốt lá và bảng FLP

- o 1 tương đương a $\Rightarrow U_a = \text{FLP}(1) = \{4,5,6\}$ (Ra 1 tập khác ta đặt là $\{B\}$)
o 2 tương đương b $\Rightarrow U_b = \text{FLP}(2) = \{3\}$ (đặt là $\{C\}$)

Xét $\{B\} = \{4,5,6\}$:

- o 4 tương đương a $\Rightarrow U_a = \text{FLP}(4) = \{4,5,6\}$ (là $\{B\}$)
- o 5,6 tương đương b $\Rightarrow U_b = \text{FLP}(5) \cup \text{FLP}(6) = \{4,5,6,7\}$ (đặt là $[D]$)

52

Bảng truyền

Ədəd a		Ədəd b
A	B	C
B	B	D
C	B	
D	E	D
E	B	D

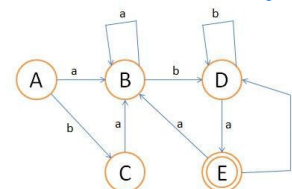
Bảng truyền

53

VỀ DFA

- ❑ Vẽ DFA theo các trạng thái ta có được (A, B, C, D, E):

- A là trạng thái bắt đầu
- Trạng thái nào chứa 8 sẽ là trạng thái kết thúc
- Đường đi dựa vào dữ liệu ta đã xét trên kia cho từng trạng thái (đọc a, đọc b)



VỀ DFA

54



Xây dựng DFA trực tiếp từ biểu thức chính quy

➤ Xây dựng DFA trực tiếp từ biểu thức chính quy:

- Vẽ cây phân tích
- Tính các giá trị Nullable, First Post (FP), Last Post (LP).
- Lập bảng Follow Post (FLP)
- Tìm tập các trạng thái, bảng truyền
- Vẽ DFA

➤ Tối thiểu trạng thái của DFA.

55



Tối thiểu số trạng thái của DFA

- ❑ Khái niệm DFA đầy đủ, trạng thái chết d
- ❑ Chuỗi w phân biệt trạng thái s với trạng thái t
- ❑ Giải thuật 3.6: Tối thiểu trạng thái của DFA

Nhập: DFA, gọi là M có S, Σ , s0, F. M là DFA đầy đủ.

Xuất: DFA, gọi là M' chấp nhận ngôn ngữ như M, với số trạng thái nhỏ nhất.

❑ Phương pháp:

1. Tạo phần khởi đầu Π có 2 nhóm: các trạng thái kết thúc F, và các trạng thái không kết thúc S - F.
2. Áp dụng thủ tục (mô phỏng 3.6) để tạo Π^{new}
3. Nếu $\Pi^{new} = \Pi$ thì $\Pi^{final} = \Pi$ tiếp tục bước 4, ngược lại lặp lại bước 2, với $\Pi = \Pi^{new}$
4. Chọn mỗi nhóm 1 trạng thái đại diện và đó là trạng thái của M'.
5. Nếu M' có trạng thái chết d thì loại nó ra khỏi M'. Tất cả các sự truyền đến trạng thái d đều không xác định.

56



Tối thiểu số trạng thái của DFA

Mô phỏng 3.6: Giải thuật tạo Π^{new}

for với mỗi nhóm G của Π do begin

- Chia G thành các nhóm nhỏ hơn sao cho hai trạng thái s và t của G sẽ ở cùng một nhóm nhỏ hơn nếu và chỉ nếu các sự truyền trên tất cả các ký hiệu nhập a từ s và t đều đi đến các trạng thái kế tiếp ở trong cùng một nhóm của Π

- Ta thay G bằng các nhóm nhỏ hơn vừa được tạo nên, cho chúng vào Π^{new}

end

57



Ví dụ đơn giản DFA

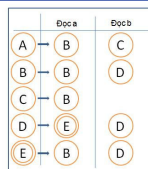
58



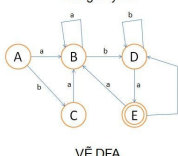
Tối thiểu số trạng thái của DFA

- ❑ $\Pi = \{(E), (ABCD)\}$
- ❑ Xét (ABCD) trên sự truyền a
 - A $\rightarrow$ B : thuộc (ABCD)
 - B $\rightarrow$ B :
 - C $\rightarrow$ B :
 - D $\rightarrow$ E : không thuộc (ABCD) $\Rightarrow$ tách D riêng
- ❑ Tương tự trên b
- ❑ $\Pi^{new} = \{(E), (ABC), (D)\}$

Thông thường những trạng thái rút gọn được là có sự truyền đến cùng một trạng thái trên một ký hiệu nhập và đến chính mình trên ký hiệu nhập khác.



Bảng truyền



Vẽ DFA

Khoa KTMT - UIT

TS. Vũ Đức Lung

59



CÂU HỎI VÀ BÀI TẬP CHƯƠNG 03

Khoa KTMT

Vũ Đức Lung

60



Chương 04:

Văn phạm phi ngữ cảnh (Context Free Grammar)

Nội dung:

- Văn phạm phi ngữ cảnh (CFG)
- Giảm lược văn phạm phi ngữ cảnh
- Chuẩn hóa văn phạm phi ngữ cảnh
- Các tính chất của văn phạm phi ngữ cảnh

1



NHẬN XÉT

- CFG là những ký hiệu cho việc mô tả ngôn ngữ
- CFG mạnh hơn FA hay RE, nhưng vẫn không thể mô tả mọi ngôn ngữ
- Thường được dùng để mô tả những cấu trúc lồng vào nhau
- Ý tưởng cơ bản là dùng những biến thay thế cho tập các chuỗi (hay ngôn ngữ)
- Các biến này được định nghĩa hồi qui với các biến khác và các định nghĩa này gọi là các qui tắc sinh, hay luật sinh

2



Ví dụ: CFG for $\{0^n1^n \mid n \geq 1\}$

- Luật sinh (Productions) có dạng:
 - $S \rightarrow 01$
 - $S \rightarrow 0S1$
- Chuỗi 01 là một ngôn ngữ
- Phương pháp quy nạp (Induction): Nếu w thuộc ngôn ngữ thì $0w1$ cũng thuộc ngôn ngữ.

3



Văn phạm phi ngữ cảnh

Định nghĩa: là hệ thống gồm 4 thành phần $G(V, T, P, S)$

- V (*Variables* = *nonterminals*): tập hữu hạn các biến (ký tự chưa kết thúc)
- T (*Terminals*): tập hữu hạn các ký tự kết thúc ($V \cap T = \emptyset$)
- P (*Productions*): tập hữu hạn các luật sinh dạng $A \rightarrow \alpha$ ($\alpha \in (V \cup T)^*$)
- S (*Start symbol*): ký hiệu bắt đầu của văn phạm

Quy ước:

- V : chữ in hoa ($A, B, C, \dots$); T : chữ in thường ($a, b, c, \dots, w, x, y, \dots$)
- $\alpha, \beta, \gamma, \dots$ biểu diễn chuỗi ký hiệu kết thúc và biến

4



Ví dụ: Formal CFG

- A formal CFG for $\{0^n1^n \mid n \geq 1\}$.

- Terminals = $\{0, 1\}$.
- Variables = $\{S\}$.
- Start symbol = S .
- Productions =

$S \rightarrow 01$

$S \rightarrow 0S1$

- $G = (\{S, A, B\}, \{a, b\}, P, S)$ với P gồm các luật sinh:

$S \rightarrow AB$

$A \rightarrow aA$

$A \rightarrow a$

$B \rightarrow bB$

$B \rightarrow b$

hay

$S \rightarrow AB$

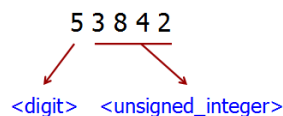
$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

5



Ví dụ số nguyên không dấu (Unsigned Integers)



- Ký hiệu bắt đầu $\langle \text{unsigned_integer} \rangle$
- Ký hiệu không kết thúc $\langle \text{unsigned_integer} \rangle, \langle \text{digit} \rangle$
- Ký hiệu kết thúc $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$
- Luật
 - $\langle \text{unsigned_integer} \rangle \rightarrow \langle \text{digit} \rangle$
 - $\langle \text{unsigned_integer} \rangle \rightarrow \langle \text{digit} \rangle \langle \text{unsigned_integer} \rangle$

6



BNF (Backus-Naur Form)

- ❑ 1959, John Backus giới thiệu ALGOL 58 thuộc nhóm ACM-GAMM
- ❑ 1960 Peter Naur cho ra phiên bản mới ALGOL 60
- ❑ BNF là một ký hiệu tự nhiên mô tả cú pháp, là một siêu ngôn ngữ mô tả ngôn ngữ khác
- ❑ Các Grammar cho ngôn ngữ lập trình thường được viết dưới dạng BNF

VD:

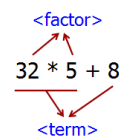
$\langle \text{unsigned_integer} \rangle ::= \langle \text{digit} \rangle \mid \langle \text{digit} \rangle \langle \text{unsigned_integer} \rangle$

7



Ví dụ một biểu thức

- Start symbol $\langle \text{expression} \rangle$
- Non-terminals $\langle \text{expression} \rangle, \langle \text{term} \rangle, \langle \text{factor} \rangle, \langle \text{unsigned_integer} \rangle, \langle \text{term_op} \rangle, \langle \text{factor_op} \rangle$
- Terminals $0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /$



- Productions:
 - $\langle \text{expression} \rangle \rightarrow \langle \text{term} \rangle \mid \langle \text{expression} \rangle \langle \text{term_op} \rangle \langle \text{term} \rangle$
 - $\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle \mid \langle \text{term} \rangle \langle \text{factor_op} \rangle \langle \text{factor} \rangle$
 - $\langle \text{factor} \rangle \rightarrow \langle \text{unsigned_integer} \rangle \mid (\langle \text{expression} \rangle)$
 - $\langle \text{term_op} \rangle \rightarrow + \mid -$
 - $\langle \text{factor_op} \rangle \rightarrow * \mid /$

8



BNF mở rộng (Extended BNF)

- ❑ Không làm tăng khả năng đặc tả của BNF, chỉ là cách biểu diễn gọn hơn
- 1. Phần lựa chọn được đặt vào trong dấu []
 $\langle \text{selection} \rangle \rightarrow \text{if} (\langle \text{expression} \rangle) \langle \text{statement} \rangle [\text{else} \langle \text{statement} \rangle]$
- 2. Đặt những phần tương tự trong RHS vào trong () và ngăn cách bởi |
 $\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle (+ \mid -) \text{const}$
- 3. Đặt những phần lặp lại trong {}
 $\langle \text{ident} \rangle \rightarrow \text{letter} \{ \text{letter} \mid \text{digit} \}$
- ❑

9



BNF vs EBNF

BNF:

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$
 $\mid \langle \text{expr} \rangle - \langle \text{term} \rangle$
 $\mid \langle \text{term} \rangle$
 $\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle$
 $\mid \langle \text{term} \rangle / \langle \text{factor} \rangle$
 $\mid \langle \text{factor} \rangle$

EBNF:

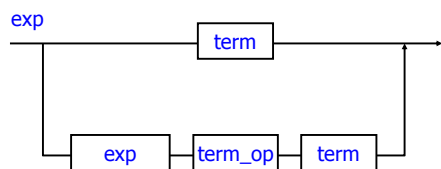
$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle \{ (+ \mid -) \langle \text{term} \rangle \}$
 $\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle \{ (* \mid /) \langle \text{factor} \rangle \}$

10



Sơ đồ cú pháp (Syntax Diagrams)

Sơ đồ cú pháp của biểu thức



11



Dẫn xuất và ngôn ngữ (Derivation and Language)

Dẫn xuất:

- Nếu $A \rightarrow \beta$ là luật sinh trong văn phạm G và α, γ là 2 chuỗi bất kỳ, thì khi áp dụng luật sinh $A \rightarrow \beta$ vào chuỗi $\alpha A \gamma$ ta sẽ thu được chuỗi $\alpha \beta \gamma$:

$$\alpha A \gamma \Rightarrow_G \alpha \beta \gamma$$

- Giả sử: $\alpha_1 \Rightarrow_G \alpha_2, \alpha_2 \Rightarrow_G \alpha_3, \dots, \alpha_{m-1} \Rightarrow_G \alpha_m$, ta có:

$$\alpha_1 \Rightarrow_G^* \alpha_m$$

- Ta có: $\alpha \Rightarrow_G^* \alpha$ với mọi chuỗi α
- Thông thường, ta sẽ dùng $\Rightarrow$ và $\Rightarrow^*$ thay cho $\Rightarrow_G$ và $\Rightarrow_G^*$

Ngôn ngữ sinh bởi CFG: cho CFG $G(V, T, P, S)$

$$L(G) = \{ w \mid w \in T^* \text{ và } S \Rightarrow_G^* w \}$$

(chuỗi w gồm toàn ký hiệu kết thúc và được dẫn ra từ S)

12



Cây dẫn xuất

Định nghĩa: cây dẫn xuất (hay cây phân tích cú pháp) của một văn phạm $G(V, T, P, S)$ có đặc điểm

- (1) Mỗi nút có một nhãn, là một ký hiệu $\in (V \cup T \cup \{\epsilon\})$
- (2) Nút gốc có nhãn là S (ký hiệu bắt đầu)
- (3) Nếu nút trung gian có nhãn A thì $A \in V$
- (4) Nếu nút n có nhãn A và các đỉnh $n_1, n_2, \dots, n_k$ là con của n theo thứ tự từ trái sang phải có nhãn lần lượt là $X_1, X_2, \dots, X_k$ thì $A \rightarrow X_1 X_2 \dots X_k$ là một luật sinh trong P
- (5) Nếu nút n có nhãn là ϵ thì n phải là nút lá và là nút con duy nhất của nút cha của nó

ĐỊNH LÝ 1: Nếu $G(V, T, P, S)$ là một văn phạm phi ngữ cảnh thì $S \Rightarrow^* \alpha$ nếu và chỉ nếu có cây dẫn xuất trong văn phạm sinh ra α .

13



Dẫn xuất trái nhất - Dẫn xuất phải nhất

Dẫn xuất trái nhất (phải nhất): nếu tại mỗi bước dẫn xuất, luật sinh được áp dụng vào biến bên trái nhất (phải nhất)

Ví dụ: xét văn phạm G với luật sinh:

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aA \mid a \\ B &\rightarrow bB \mid b \end{aligned}$$

- Các dẫn xuất khác nhau cho từ **aaabb**:

- (a) $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaaB \Rightarrow aaabB \Rightarrow aaabb$
- (b) $S \Rightarrow AB \Rightarrow AbB \Rightarrow Abb \Rightarrow aAbb \Rightarrow aaAbb \Rightarrow aaabb$
- (c) $S \Rightarrow AB \Rightarrow aAB \Rightarrow aAbB \Rightarrow aAbb \Rightarrow aaAbb \Rightarrow aaabb$
- (d) $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaAbB \Rightarrow aaabb$

- Dẫn xuất (a) là dẫn xuất trái nhất, (b) là dẫn xuất phải nhất
- Các dẫn xuất tuy khác nhau, nhưng có cùng một cây dẫn xuất

14



Chuỗi dẫn xuất (Derivations)

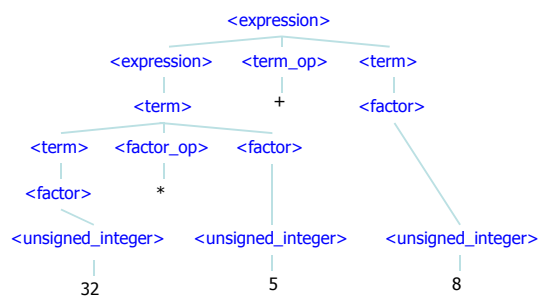
Chuỗi dẫn xuất của biểu thức: **32*5 + 8**

```
<expression>
=> <expression> <term_op> <term>
=> <term> + <factor>
=> <term> <factor_op> <factor> + <unsigned_integer>
=> <factor> * <unsigned_integer> + 4
=> <unsigned_integer> * 5 + 8
=> 32 * 5 + 8
```

15



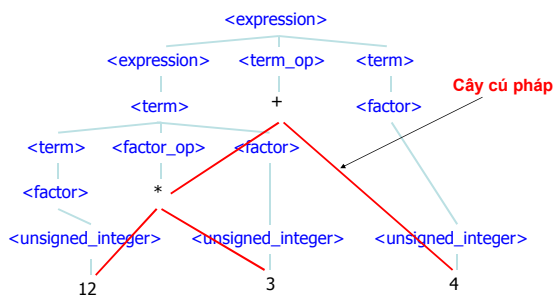
Cây phân tích cú pháp (Parse Trees)



16



Cây cú pháp (Parse Trees)



17



Sự nhập nhằng (Ambiguity)

- Văn phạm cho câu lệnh gán đơn giản

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{id} \rangle \rightarrow A|B|C$

$\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle \mid \langle \text{id} \rangle * \langle \text{expr} \rangle \mid (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

- VD biểu thức gán: $A = B * (A + C)$ có chuỗi dẫn xuất cực tả:

```
<assign> => <id> = <expr>
=> A = <expr>
=> A = <id> * <expr>
=> A = B * <expr>
=> A = B * ( <expr> )
=> A = B * ( <id> + <expr> )
=> A = B * ( A + <expr> )
=> A = B * ( A + <id> )
=> A = B * ( A + C )
```

18



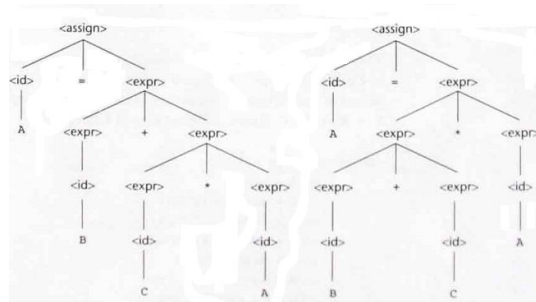
Sự nhập nhằng (Ambiguity)

- ❑ Văn phạm cho câu lệnh gán đơn giản (mở rộng VD trước)
 - $\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$
 - $\langle \text{id} \rangle \rightarrow A|B|C$
 - $\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{expr} \rangle \mid \langle \text{expr} \rangle * \langle \text{expr} \rangle \mid (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$
- ❑ Là một văn phạm nhập nhằng vì $A = B + C * A$ có 2 cây cú pháp

19



Sự nhập nhằng (Ambiguity)



20



Sự nhập nhằng (Ambiguity)

Khắc phục văn phạm nhập nhằng:

- Quy định rằng các phép cộng và nhân luôn được thực hiện theo thứ tự từ trái sang phải (trừ khi gặp ngoặc đơn)

$$E \rightarrow E + T \mid E * T \mid T$$

$$T \rightarrow (E) \mid a$$

- Quy định rằng khi không có dấu ngoặc đơn ngăn cách thì phép nhân luôn được thực hiện ưu tiên hơn phép cộng

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid a$$

21



Giảm lược văn phạm phi ngữ cảnh

Trong CFG có thể chứa các yếu tố thừa:

- Các ký hiệu không tham gia vào quá trình dẫn xuất ra chuỗi ký hiệu kết thúc
- Luật sinh dạng $A \rightarrow B$ (làm kéo dài chuỗi dẫn xuất)

⇒ giảm lược văn phạm nhằm loại bỏ những yếu tố vô ích, nhưng không được làm thay đổi khả năng sản sinh ngôn ngữ của văn phạm

- Mỗi biến và mỗi ký hiệu kết thúc của văn phạm đều xuất hiện trong dẫn xuất của một số chuỗi trong ngôn ngữ
- Không có luật sinh $A \rightarrow B$ (với A, B đều là biến)
- Nếu ngôn ngữ không chấp nhận chuỗi rỗng ϵ thì không cần luật sinh $A \rightarrow \epsilon$.

22



Các ký hiệu vô ích

Khái niệm: một ký hiệu X được gọi là có ích nếu có một dẫn xuất

$$S \Rightarrow^* \alpha X \beta \Rightarrow^* w$$

với α, β là các chuỗi bất kỳ và $w \in T^*$.

⇒ có 2 đặc điểm cho ký hiệu có ích

- X phải dẫn ra chuỗi ký hiệu kết thúc
- X phải nằm trong dẫn xuất từ S

23



Các ký hiệu vô ích

Bổ đề 1: (loại bỏ các biến không dẫn ra chuỗi ký hiệu kết thúc)

Cho CFG $G(V, T, P, S)$ với $L(G) \neq \emptyset$, có một CFG $G'(V', T', P', S)$ tương đương sao cho mỗi $A \in V'$ tồn tại $w \in T^*$ để $A \Rightarrow^* w$

Giải thuật tìm V' :

Begin

- (1) $\text{Old}V' := \emptyset;$
- (2) $\text{New}V' := \{ A \mid A \rightarrow w \text{ với } w \in T^* \};$
- (3) **While** $\text{Old}V' \neq \text{New}V'$ **do**
 begin
 (4) $\text{Old}V' := \text{New}V';$
 (5) $\text{New}V' := \text{Old}V' \cup \{ A \mid A \rightarrow \alpha \text{ với } \alpha \in (T \cup \text{Old}V')^* \}$
 end;
 (6) $V' := \text{New}V';$

End;

24



Các ký hiệu vô ích

Bổ đề 2: (Loại bỏ các biến không được dẫn ra từ ký hiệu bắt đầu)

Cho CFG $G(V, T, P, S)$, ta có thể tìm được CFG $G'(V', T', P', S)$ tương đương sao cho mỗi $X \in (V' \cup T')$ tồn tại $\alpha, \beta \in (V \cup T)^*$ để $S \Rightarrow^* \alpha X \beta$

Cách tìm:

- Đặt $V' = \{S\}$
- Nếu $A \in V'$ và $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ là các luật sinh trong P thì
 - Thêm các biến của $\alpha_1, \alpha_2, \dots, \alpha_n$ vào V'
- Lặp lại cho đến khi không còn biến nào được thêm vào nữa

25



Các ký hiệu vô ích

Định lý 2: mỗi ngôn ngữ phi ngữ cảnh (CFL) không rỗng được sinh ra từ một văn phạm phi ngữ cảnh (CFG) không có ký hiệu vô ích

Ví dụ: xét văn phạm $S \rightarrow AB \mid a$
 $A \rightarrow a$

- **Áp dụng bổ đề 1:** loại bỏ $S \rightarrow AB$ ta còn G :
 $S \rightarrow a$
 $A \rightarrow a$
- **Áp dụng bổ đề 2:** loại bỏ luật sinh thứ hai
→ G tương đương = $(\{S\}, \{a\}, \{S \rightarrow a\}, S)$

26



Luật sinh ϵ

Định lý 3: (Loại bỏ luật sinh $A \rightarrow \epsilon$)

Cho CFG $G(V, T, P, S)$ và L là ngôn ngữ sinh ra bởi G . Khi đó $L - \{\epsilon\}$ là ngôn ngữ sinh ra bởi CFG $G'(V, T, P', S)$ không có ký hiệu vô ích và không có luật sinh ϵ .

Cách tìm:

- **Bước 1:** xác định tập biến rỗng Nullable
 - i. $A \rightarrow \epsilon \Rightarrow A \in \text{Nullable}$
 - ii. $B \rightarrow X_1 X_2 \dots X_n, \forall X_i \in \text{Nullable} \Rightarrow B \in \text{Nullable}$
- **Bước 2:** xây dựng tập luật sinh P'
Với mỗi luật sinh $A \rightarrow X_1 X_2 \dots X_n$ trong P , ta xây dựng luật sinh $A \rightarrow \alpha_1 \alpha_2 \dots \alpha_n$ với điều kiện:
 - i. Nếu $X_i \notin \text{Nullable}$ thì $\alpha_i = X_i$
 - ii. Nếu $X_i \in \text{Nullable}$ thì $\alpha_i = X_i \mid \epsilon$
 - iii. Không phải tất cả α_i đều bằng ϵ

27



VD loại bỏ luật sinh ϵ

$S \rightarrow ABC, A \rightarrow aA \mid \epsilon, B \rightarrow bB \mid \epsilon, C \rightarrow \epsilon$

□ A, B, C , and S are all nullable.

□ New grammar:

$S \rightarrow \cancel{ABC} \mid \cancel{AB} \mid \cancel{A}B \mid \cancel{A} \mid \cancel{B} \mid \cancel{C}$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

Note: C is now useless.
Eliminate its productions.

28



Luật sinh ϵ

Ví dụ: loại bỏ luật sinh ϵ trong văn phạm sau:

$S \rightarrow AB$
 $A \rightarrow aA \mid \epsilon$
 $B \rightarrow bB \mid \epsilon$

- **Bước 1:** xác định tập biến rỗng Nullable

- i. $A \rightarrow \epsilon \Rightarrow A \in \text{Nullable}$
- ii. $B \rightarrow \epsilon \Rightarrow B \in \text{Nullable}$
- iii. $S \rightarrow AB \Rightarrow S \in \text{Nullable}$

- **Bước 2:** xây dựng tập luật sinh P'

$S \rightarrow AB \mid A\epsilon \mid \epsilon B$
 $A \rightarrow aA \mid a\epsilon$
 $B \rightarrow bB \mid b\epsilon$

Chú ý: văn phạm G' không chấp nhận chuỗi rỗng ϵ như văn phạm G . Để G' tương đương G , ta cần thêm luật sinh $S \rightarrow \epsilon$ vào G' .

29



Luật sinh đơn vị

Một luật sinh có dạng $A \rightarrow B$ với A, B đều là biến gọi là **luật sinh đơn vị**.

Định lý 4: (Loại bỏ luật sinh $A \rightarrow B$)

Mỗi CFL không chứa ϵ được sinh ra bởi CFG không có ký hiệu vô ích, không có luật sinh ϵ hoặc luật sinh đơn vị.

Cách tìm: đặt $L=L(G)$ là CFL không chứa ϵ và được sinh ra bởi văn phạm $G(V, T, P, S)$. Theo định lý 3, ta có thể loại bỏ tất cả luật sinh ϵ trong G .

Để loại bỏ luật sinh đơn vị, ta xây dựng tập P' mới theo giải thuật:

For (mỗi biến $A \in V$) **do**
 Begin
 Tính $\Delta_A = \{B \mid B \in V \text{ và } A \Rightarrow^* B\}$;
 For (mỗi biến $B \in \Delta_A$) **do**
 For (mỗi luật sinh $B \rightarrow \alpha$ thuộc P) **do**
 If ($B \rightarrow \alpha$ không là luật sinh đơn vị) **then**
 Thêm luật sinh $A \rightarrow \alpha$ vào P'
 End

30



Luật sinh đơn vị

Ví dụ: loại bỏ luật sinh đơn vị trong văn phạm

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

Ta có: $\Delta_E = \{E, T, F\} \Rightarrow$ thêm vào P' các luật sinh

$$E \rightarrow E + T \mid T * F \mid (E) \mid a$$

Tương tự:

$$\Delta_T = \{T, F\} \Rightarrow \text{thêm vào } P' : T \rightarrow T * F \mid (E) \mid a$$

$$\Delta_F = \{F\} \Rightarrow \text{thêm vào } P' : F \rightarrow (E) \mid a$$

31



Dạng chuẩn Chomsky (CNF) Chomsky Normal Form

□ Một CFG được gọi là **Chomsky Normal Form** nếu mỗi luật sinh có một trong hai dạng sau:

1. $A \rightarrow BC$ (right side is two variables).
2. $A \rightarrow a$ (right side is a single terminal).

ĐỊNH LÝ 5: (Dạng chuẩn Chomsky, hay CNF)

Một ngôn ngữ phi ngữ cảnh bất kỳ không chứa ϵ đều được sinh ra bằng một văn phạm nào đó mà các luật sinh có dạng $A \rightarrow BC$ hoặc $A \rightarrow a$, với A, B, C là biến còn a là ký hiệu kết thúc.

32



Dạng chuẩn Chomsky (CNF)

Ví dụ: tìm văn phạm có dạng CNF tương đương văn phạm sau:

$$\begin{aligned} S &\rightarrow A \mid ABA \\ A &\rightarrow aA \mid a \mid B \\ B &\rightarrow bB \mid b \end{aligned}$$

Bước 1: $\Delta_s = \{S, A, B\}$, $\Delta_A = \{A, B\}$, $\Delta_B = \{B\}$

$$\begin{aligned} S &\rightarrow aA \mid a \mid bB \mid b \mid ABA \\ A &\rightarrow aA \mid a \mid bB \mid b \\ B &\rightarrow bB \mid b \end{aligned}$$

Bước 2: thay a bằng C_a và b bằng C_b trong các luật sinh có độ dài vế phải > 1 :

$$\begin{aligned} S &\rightarrow C_a A \mid a \mid C_b B \mid b \mid ABA \\ A &\rightarrow C_a A \mid a \mid C_b B \mid b \\ B &\rightarrow C_b B \mid b \\ C_a &\rightarrow a \\ C_b &\rightarrow b \end{aligned}$$

33



Dạng chuẩn Chomsky (CNF)

Bước 3: thay thế các luật sinh có độ dài vế phải > 2 :

$$\begin{aligned} S &\rightarrow C_a A \mid a \mid C_b B \mid b \mid AD_1 \\ A &\rightarrow C_a A \mid a \mid C_b B \mid b \\ B &\rightarrow C_b B \mid b \\ C_a &\rightarrow a \\ C_b &\rightarrow b \\ D_1 &\rightarrow BA \end{aligned}$$

34



Dạng chuẩn Greibach (GNF)

Định lý 6: mỗi CFL bất kỳ không chứa ϵ được sinh ra bởi một CFG mà mỗi luật sinh có dạng $A \rightarrow a\alpha$ với A là biến, a là ký hiệu kết thúc và α là một chuỗi các biến (có thể rỗng)

Đặt G là CFG sinh ra CFL không chứa ϵ

Bước 1: xây dựng G' có dạng CNF tương đương G

Bước 2: đổi tên các biến trong G' thành $A_1, A_2, \dots, A_m$ ($m \geq 1$) với A_1 là ký hiệu bắt đầu. Đặt $V = \{A_1, A_2, \dots, A_m\}$

Bước 3: thay thế luật sinh sao cho nếu $A_i \rightarrow A_j \gamma$ thì $j > i$

- Nếu $j < i$: áp dụng bổ đề 3. Nếu $i = j$: áp dụng bổ đề 4 (giải thuật)
- Trong P chỉ chứa các luật sinh dạng: $A_i \rightarrow A_j \gamma$ ($j > i$), $A_i \rightarrow a \gamma$ hoặc $B_k \rightarrow \gamma$ với $\gamma \in (V \cup \{B_1, B_2, \dots, B_{i-1}\})^*$

Bước 4: thay thế các A_i – luật sinh về đúng dạng (áp dụng bổ đề 3)

Bước 5: thay thế các B_k – luật sinh về đúng dạng (bổ đề 3)

35



Dạng chuẩn Greibach (GNF)

Giải thuật: (thay thế sao cho $A_i \rightarrow A_j \gamma$ thì $j > i$)

Begin

- (1) **for** $k := 1$ **to** m **do begin**
- (2) **for** $j := 1$ **to** $k-1$ **do**
- (3) **for** Mỗi luật sinh dạng $A_k \rightarrow A_j \alpha$ **do**
begin
- (4) **for** Tất cả luật sinh $A_j \rightarrow \beta$ **do**
- (5) Thêm luật sinh $A_k \rightarrow \beta \alpha$;
- (6) Loại bỏ luật sinh $A_k \rightarrow A_j \alpha$
end;
- (7) **for** Mỗi luật sinh dạng $A_k \rightarrow A_k \alpha$ **do**
begin
- (8) Thêm các luật sinh $B_k \rightarrow \alpha$ và $B_k \rightarrow \alpha B_k$;
- (9) Loại bỏ luật sinh $A_k \rightarrow A_k \alpha$
end;
- (10) **for** Mỗi luật sinh $A_k \rightarrow \beta$ trong đó β không bắt đầu bằng A_k **do**
- (11) Thêm luật sinh $A_k \rightarrow \beta B_k$
end;

end;

36



Dạng chuẩn Greibach (GNF)

Ví dụ: tìm văn phạm có dạng GNF cho văn phạm G sau:

$$\begin{array}{l} A_1 \rightarrow A_2 A_1 \mid A_2 A_3 \\ A_2 \rightarrow A_3 A_1 \mid a \\ A_3 \rightarrow A_2 A_2 \mid b \end{array}$$

Bước 1: G thỏa CNF

Bước 2: ta có $V = \{A_1, A_2, A_3\}$

Bước 3: ta cần sửa đổi luật sinh $A_3 \rightarrow A_2 A_2$

- Áp dụng bổ đề 3: $A_3 \rightarrow A_3 A_1 A_2 \mid a A_2$
 $A_3 \rightarrow A_3 A_1 A_2 \mid a A_2 \mid b$
- Áp dụng bổ đề 4, ta thu được tập luật sinh:

$$\begin{array}{l} A_1 \rightarrow A_2 A_1 \mid A_2 A_3 \\ A_2 \rightarrow A_3 A_1 \mid a \\ A_3 \rightarrow a A_2 \mid b \mid a A_2 B \mid b B \\ B \rightarrow A_1 A_2 \mid A_1 A_2 B \end{array}$$

37



Dạng chuẩn Greibach (GNF)

Bước 4: A_3 đã có dạng chuẩn. Thay thế A_3 vào A_2 :

$$\begin{array}{l} B \rightarrow A_1 A_2 \mid A_1 A_2 B \\ A_3 \rightarrow a A_2 \mid b \mid a A_2 B \mid b B \\ A_2 \rightarrow a A_2 A_1 \mid b A_1 \mid a A_2 B A_1 \mid b B A_1 \mid a \\ A_1 \rightarrow a A_2 A_1 A_1 \mid b A_1 A_1 \mid a A_2 B A_1 A_1 \mid b B A_1 A_1 \mid a A_1 \mid \\ a A_2 A_1 A_3 \mid b A_1 A_3 \mid a A_2 B A_1 A_3 \mid b B A_1 A_3 \mid a A_3 \end{array}$$

Bước 5: thay thế các B_k – luật sinh

$$\begin{array}{l} B \rightarrow a A_2 A_1 A_1 A_2 \mid b A_1 A_1 A_2 \mid a A_2 B A_1 A_1 A_2 \mid b B A_1 A_1 A_2 \mid a A_1 A_2 \mid \\ a A_2 A_1 A_2 A_2 \mid b A_1 A_3 A_2 \mid a A_2 B A_1 A_3 A_2 \mid b B A_1 A_3 A_2 \mid a A_3 A_2 \mid \\ a A_2 A_1 A_1 A_2 B \mid b A_1 A_1 A_2 B \mid a A_2 B A_1 A_1 A_2 B \mid b B A_1 A_1 A_2 B \mid a A_1 A_2 B \mid \\ a A_2 A_1 A_3 A_2 B \mid b A_1 A_3 A_2 B \mid a A_2 B A_1 A_3 A_2 B \mid b B A_1 A_3 A_2 B \mid \\ a A_3 A_2 B \end{array}$$

38



Bổ đề bơm cho CFL

Bổ đề bơm: cho L là một CFL bất kỳ, tồn tại một số n chỉ phụ thuộc vào L sao cho nếu $z \in L$ và $|z| \geq n$ thì ta có thể viết $z = uvwxy$ sao cho: $|vx| \geq 1$, $|vwx| \leq n$ và $\forall i \geq 0$ ta có $uv^iwx^iy \in L$

Ví dụ: chứng minh $L = \{a^i b^j c^k \mid i \geq 1\}$ không là CFL

- Giả sử L là CFL, khi đó tồn tại số n theo bổ đề bơm
- Xét chuỗi $z = a^n b^n c^n$, $|z| \geq n$, ta có thể viết $z = uvwxy$ thỏa bổ đề
- Ta có: $vwx \in a^n b^n c^n$, $|vwx| \leq n$ nên vwx không thể đồng thời chứa cả ký hiệu a và c (vì giữa a và c có n ký hiệu b) $\rightarrow vx$ cũng không thể chứa cả ký hiệu a và c.
- Do $|vx| \geq 1$ và trong $uvwxy$ chứa số ký hiệu a, b, c bằng nhau:
 - Nếu vx có chứa ký hiệu a (nên không thể chứa ký hiệu c) thì khi bơm chuỗi vx , số ký hiệu c sẽ không đổi (luôn là n), nhưng số ký hiệu a sẽ thay đổi. Ví dụ: chuỗi $uv^0wx^0y \notin L$ vì có số ký hiệu a (ít hơn n) số ký hiệu c (luôn là n) không bằng nhau.
 - Nếu vx không chứa ký hiệu a thì khi bơm chuỗi vx , số ký hiệu a không đổi, nhưng số ký hiệu b (hoặc c) sẽ thay đổi.

41



Tính chất đóng của CFL

Định lý 5.7: CFL đóng với phép hợp, phép kết nối và phép bao đóng Kleen.

Định lý 5.8: CFL không đóng với phép giao

Hệ quả: CFL không đóng với phép lấy phần bù

40



CÂU HỎI VÀ BÀI TẬP CHƯƠNG 04